



Diplomado en Data Science Aplicada con Python para la Toma de Decisiones

Clase 3: pandas y Carga de Datos — Campo Volve



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos y Qué Viene Hoy

✔ Ya sabemos:

- ✔ Variables, tipos, `input()` (Clase 1)
- ✔ `if` / `elif` / `else` (Clase 2)
- ✔ Listas y listas paralelas (Clase 2)
- ✔ Ciclos `for`: acumular, contar, filtrar (Clase 2)

El reporte de la batería (5 pozos) nos tomó ~25 líneas de código...

▶▶ Hoy:

- Qué son los **datos** y los **datasets**
- Qué son las **librerías** y cómo se instalan (`pip`)
- **pandas**: tablas de datos en Python
- **Cargar archivos** reales en Colab
- **Well logs**: el formato LAS
- Operaciones y **gráficos** con pandas

*Con datos reales del campo **Volve** (Mar del Norte, 2008–2016).*

Los Datos

Qué son y dónde viven

¿Qué es un Dato? ¿Qué es un Dataset?

Un dato es un hecho registrado. Un dataset es una colección organizada de datos.

Datos sueltos:

- 🔥 “El 3 de junio de 2008 el pozo F-12 produjo 3 520 barriles.”
- ⚙️ “A 3 550 m de profundidad la roca tiene densidad 2.31.”
- 🕒 “El pozo F-14 operó 24 horas ese día.”

Un dataset:

- ✓ **Miles** de esos hechos, juntos y organizados.
- ✓ Con una **estructura** que permite buscar, comparar y calcular.
- ✓ Hoy usaremos dos datasets **reales** de un campo petrolero.

La estructura más común para organizarlos es la **tabla**.

Datos Tabulares: Filas y Columnas

Una tabla, igual que una hoja de Excel. Es la forma en que viene casi todo dato industrial.

- Cada **fila** es un **registro**: un día de un pozo, una medición.
- ↓ Cada **columna** es una **variable**: fecha, producción, presión.
- La **celda** es el cruce: un valor concreto.

una tabla de producción

fecha	pozo	oil
2008-06-01	15/9-F-12	3520
↪ fila		
2008-06-01	15/9-F-14	2110
2008-06-02	15/9-F-12	3480
columna	columna	columna

💡 En la Clase 2 esto eran **listas paralelas**: una lista por columna. Hoy la tabla será un solo objeto.

¿Dónde Viven los Datos? Los Archivos

Los datos se guardan en archivos. El formato más universal: el CSV.

- **CSV** = *Comma-Separated Values*: valores separados por comas.
- A Es **texto plano**: se abre hasta con el Bloc de notas.
- A La **primera línea** trae los nombres de las columnas.
- ≡ Cada línea siguiente es **una fila**.

■ produccion.csv (por dentro)

```
fecha,pozo,oil  
2008-06-01,15/9-F-12,3520  
2008-06-01,15/9-F-14,2110  
2008-06-02,15/9-F-12,3480
```

Existen otros formatos (Excel, JSON, LAS...). Hoy empezamos por el más simple y al final de la clase veremos uno especializado de la industria petrolera.

Librerías

Las cajas de herramientas de Python

¿Qué es una Librería?

Código que otra gente ya escribió, probó y regala, listo para que lo usemos.

- 🔧 Como una **caja de herramientas**: no fabricamos el martillo, lo tomamos y usamos.
- 👥 Las mantienen comunidades de miles de desarrolladores en el mundo.
- 🐍 Python es popular en gran parte por su **ecosistema** de librerías.

Las que usaremos hoy:

Librería	Para qué
pandas	tablas de datos
lasio	leer well logs (LAS)

*Antes de usar una librería hay que hacer dos cosas:
instalarla (una vez) e **importarla** (en cada cuaderno).*

Instalar: pip, el Gestor de Librerías

pip descarga e instala librerías desde el repositorio oficial de Python (PyPI).

 Python

```
!pip install lasio
```

>_ Output

```
Collecting lasio
Installing collected packages:
  ↳ lasio
Successfully installed lasio-0.32
```

- Piensen en pip como la **tienda de aplicaciones** de Python.
- El ! le dice a Colab: “esto es un comando del sistema, no código Python”.
- Se instala **una vez por sesión** de Colab.
- ✓ **Colab ya trae preinstaladas** las más famosas (pandas incluida). Solo instalaremos las especializadas, como lasio.

Importar: Traer la Librería al Cuaderno

Instalar la deja en el computador; importar la trae a nuestro código.

Python

```
import pandas as pd

df = pd.DataFrame(...)
```

- as pd crea un **alias**: un apodo corto para no escribir pandas . todo el tiempo.
- pd es la convención **universal**: todo el mundo la usa, úsenla también.
- Se importa **una vez por cuaderno**, normalmente en la primera celda.

Se lee: “importa pandas y llámala pd”.

💡 Regla de tres pasos: **instalar** (pip, si hace falta) → **importar** (import) → **usar**.

pandas

Tablas de datos en Python

¿Qué es pandas?

La librería de Python para trabajar con tablas de datos. La usaremos el resto del diplomado.

- ▣ Su objeto central es el **DataFrame**: una tabla con filas y columnas *con nombre*.
- ≡ Una sola columna se llama **Series**.
- ⚡ Opera sobre **columnas enteras de una vez**: adiós a los for para sumar o filtrar.

la idea

DataFrame -> toda la tabla
Series -> una columna

```
      pozo    bopd    <-  
      ↪ DataFrame  
0  F-12      3520  
1  F-14      2110  
      ^  
      una Series
```

Paréntesis: los Diccionarios

Antes de crear tablas necesitamos un tipo nuevo: el diccionario. Asocia una etiqueta a un valor.

Python

```
pozo = {  
    "nombre": "Sacha-042",  
    "bopd": 1250,  
}  
  
print(pozo["nombre"])  
print(pozo["bopd"])
```

>_ Output

```
Sacha-042  
1250
```

- Se crea con **llaves** { } y pares `clave`: valor.
- Como un diccionario de verdad: buscamos por la **palabra** (clave), no por posición.
- La lista accede por número (`lista[0]`); el diccionario, por nombre (`pozo["bopd"]`).

Lo usaremos ya mismo para nombrar las columnas de una tabla.

Nuestro Primer DataFrame

Las listas paralelas de la Clase 2 + un diccionario = una tabla.

Python

```
import pandas as pd

datos = {
    "pozo": ["Sacha-042",
            ↪ "Sacha-108",
              "Auca-63"],
    "bopd": [1250, 780, 340],
    "corte": [0.32, 0.55, 0.78],
}

df = pd.DataFrame(datos)
print(df)
```

>_ Output

	pozo	bopd	corte
0	Sacha-042	1250	0.32
1	Sacha-108	780	0.55
2	Auca-63	340	0.78

- Cada **clave** del diccionario es el **nombre de una columna**; su lista, los valores.
- El 0, 1, 2 de la izquierda es el **índice**: el número de cada fila. pandas lo agrega solo.

Su Turno: Mi Primera Tabla

🕒 5 min

Creen su propio DataFrame con datos de 3 equipos de superficie.

En el notebook:

1. Importen pandas como `pd`.
2. Armen un diccionario con tres columnas: `equipo` (bomba, separador, compresor), `horas_uso` y `estado` (texto).
3. Conviértanlo en DataFrame con `pd.DataFrame(...)`.
4. Muéstrenlo con `print()`.

💡 Recuerden: las tres listas deben tener el **mismo largo**.

Cargar Archivos en Colab

Tres formas de traer datos al cuaderno

El Dataset de Hoy: Producción del Campo Volve

Datos reales y públicos: Equinor liberó en 2018 todos los datos de su campo Volve (Mar del Norte).

- 🌐 Campo operado por Equinor (Noruega), produjo de **2008 a 2016**.
- 📄 Preparamos un **CSV sencillo** a partir del Excel original de Equinor.
- 📊 **15 634 filas:** la producción diaria de **7 pozos** durante 9 años.

Columna	Qué es
fecha	día de la medición
pozo	nombre del pozo
horas	horas en operación
oil	petróleo ($\text{Sm}^3/\text{día}$)
gas	gas ($\text{Sm}^3/\text{día}$)
agua	agua ($\text{Sm}^3/\text{día}$)

💡 Sm^3 = metros cúbicos estándar, la unidad de volumen que usa la industria en Noruega.

Método 1: Leer Desde una URL

Si el archivo está en internet, pandas lo descarga y lee en un solo paso. Es lo que haremos hoy.

Python

```
import pandas as pd

url = ".../vuelve_produccion.csv"
prod = pd.read_csv(url)

print(prod.shape)
```

>_ Output

```
(15634, 6)
```

- read_csv recibe una **ruta o URL** y devuelve un **DataFrame**.
- La URL completa está en el notebook (apunta al repositorio del curso).
- ✓ 15 634 filas y 6 columnas cargadas en segundos.

Método 2: Subir un Archivo del Computador

Para archivos que tenemos en el disco (un Excel del trabajo, un CSV que nos enviaron).

1  Click en el ícono de **carpeta** (barra izquierda de Colab)

2  Botón **Subir** y elegir el archivo

3  Leerlo por su **nombre** con `read_csv`

 Python

```
prod = pd.read_csv(  
    "vuelveproduccion.csv")
```

- ⚠ El archivo subido **se borra** cuando se cierra la sesión de Colab.
- ➔ Ahora la ruta es solo el **nombre** del archivo: ya está "adentro".

Método 3: Montar Google Drive

Para trabajar siempre con los mismos archivos, sin subirlos cada vez.

 Python

```
from google.colab import drive
drive.mount("/content/drive")

prod = pd.read_csv(
    "/content/drive/MyDrive/datos.csv")
```

- “Montar” = conectar su Drive al cuaderno. Colab pedirá permiso con su cuenta Google.
- Sus archivos aparecen bajo /content/drive/MyDrive/.
- ✓ Los archivos **persisten** entre sesiones.

💡 Los tres métodos terminan igual: `pd.read_csv(ruta)` → un DataFrame listo para trabajar.

Primer Vistazo a los Datos

Regla de oro: apenas se carga un dataset, mirarlo.

Python

```
prod.head() # primeras 5 filas
```

>_ Output

	fecha	pozo	horas	oil
0	2007-09-01	15/9-F-4	NaN	NaN
1	2007-09-01	15/9-F-5	NaN	NaN
2	2007-09-02	15/9-F-4	NaN	NaN
3	2007-09-02	15/9-F-5	NaN	NaN
4	2007-09-03	15/9-F-4	NaN	NaN

Python

```
prod.shape # (filas,  
            ↪ columnas)
```

>_ Output

```
(15634, 6)
```

? ¿Qué es ese NaN? Una celda **vacía**: en 2007 el campo aún no producía.

→ Al final de la clase aprenderemos a **limpiarlos**.

Su Turno: Cargar el Dataset

🕒 7 min

Carguen la producción de Volve y den el primer vistazo.

En el notebook (la URL está lista en la celda):

1. Leer el CSV desde la URL con `pd.read_csv(...)`.
2. Ver las primeras filas con `.head()`.
3. ¿Cuántas filas y columnas tiene? (`.shape`)
4. Probar `.tail()`: las **últimas** 5 filas. ¿De qué año son?

Los Métodos Básicos de pandas

Mirar, filtrar, resumir y graficar la tabla

Mirar la Tabla: head, tail, shape, info

Un método es una acción del DataFrame: se llama con punto, `prod.metodo()`.

Python

```
prod.head()      # primeras 5 filas
prod.tail(3)     # ultimas 3 filas
prod.shape       # (filas, columnas)
prod.info()      # resumen tecnico
```

- head y tail: una ojeada al **inicio** y al **final**. El número entre paréntesis dice cuántas filas.
- shape va **sin paréntesis**: es un dato, no una acción.
- info() muestra el **tipo** de cada columna y cuántos valores llenos tiene: miren que oil solo tiene 9161 de 15634.

```
>_ prod.info()
```

```
RangeIndex: 15634 entries
Data columns (6 columns):
 0  fecha      15634  object
 1  pozo       15634  object
 2  horas      15349  float64
 3  oil        9161   float64
```


Explorar: .describe()

Estadísticas de toda una columna en un solo comando.

Python

```
prod["oil"].describe()
```

>_ Output

```
count    9161.0
mean     1095.6
std      1323.5
min       0.0
50%      557.6
max     5901.8
```

- count: cuántos valores **no vacíos** hay.
- mean: el promedio. 50%: la mediana.
- min y max: los extremos.
- ✓ El mejor día de un pozo de Volve: **5901 Sm³** de petróleo.

Seleccionar Columnas

Igual que en el diccionario: se pide la columna por su nombre.

Python

```
prod["pozo"]           # una columna  
prod[["pozo", "oil"]]  # varias  
prod["pozo"].unique()  # sin repetir
```

>_ Output

```
'15/9-F-1 C' '15/9-F-11'  
'15/9-F-12'  '15/9-F-14'  
'15/9-F-15 D' '15/9-F-4'  
'15/9-F-5'
```

- Corchete simple []: **una** columna (una Series).
- Corchete doble [[]]: una **lista** de columnas.
- .unique(): los valores **distintos**. Aquí: los 7 pozos del campo.

Filtrar Filas: una Condición Adentro

El patrón filtro de la Clase 2 (for + if + append), ahora en una línea.

Python

```
dias_altos = prod[prod["oil"] > 4000]
print(len(dias_altos))

f12 = prod[prod["pozo"] ==
↳ "15/9-F-12"]
print(len(f12))
```

>_ Output

```
626
3056
```

- Adentro va una **condición** (como las del if de la Clase 2).
- pandas devuelve **solo las filas** donde la condición es True.
- ✓ 626 días con más de 4 000 Sm³; 3 056 días registrados del pozo F-12.

Crear una Columna Calculada

Operar columnas crea columnas nuevas. La operación se aplica a todas las filas a la vez.

Python

```
prod["liquido"] = (prod["oil"]  
                  + prod["agua"])  
  
prod[["oil", "agua",  
      ↪ "liquido"]].head(3)
```

→ Sin for: pandas suma las 15 634 filas de una vez (**vectorización**).

⚠ NaN + NaN = NaN: los huecos se **propagan**.
Otra razón para limpiarlos (ya viene).

>_ Output

```
oil    agua    liquido  
0     NaN     NaN     NaN  
1     NaN     NaN     NaN  
2     NaN     NaN     NaN
```

groupby: Agrupar y Resumir

“Divide las filas por pozo y suma el oil de cada grupo”.

Python

```
grupos = prod.groupby("pozo")
totales = grupos["oil"].sum()
print(totales)
```

>_ Output

```
pozo
15/9-F-1 C      177709.0
15/9-F-11      1147849.0
15/9-F-12      4579610.0
15/9-F-14      3942233.0
15/9-F-15 D      148519.0
15/9-F-4         0.0
15/9-F-5       41161.0
```

- En la Clase 2 esto era un acumulador por cada pozo. Aquí: `groupby + sum`.
- También existen `.mean()`, `.max()`, `.count()`.
- ✓ Dato real: **F-4 produjo 0**. Es un pozo **inyector** (mete agua al yacimiento para empujar el petróleo).

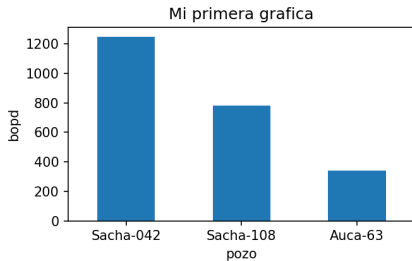
Graficar una Columna: la Idea de .plot()

Todo DataFrame sabe graficarse solo. Empecemos con la tabla pequeña de 3 pozos.

 Python

```
df.plot(x="pozo", y="bopd",  
        kind="bar")
```

- x= y y=: qué columna va en cada eje.
- kind=: el tipo de gráfico. "bar" barras, "barh" barras horizontales, "line" línea.
- ✓ Una línea de código, un gráfico. Sin librerías extra ni configuración.



Graficar con pandas: .plot()

Ahora con datos de verdad: la producción del pozo F-12 contra el tiempo.

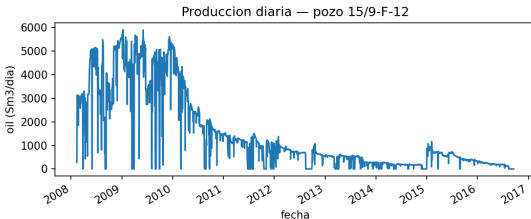
Python

```
prod["fecha"] = pd.to_datetime(
    prod["fecha"])

f12 = prod[prod["pozo"]
           == "15/9-F-12"]

f12.plot(x="fecha", y="oil")
```

- Detalle previo: `to_datetime` convierte el **texto** del CSV en **fechas de verdad**, para que el eje X ordene bien.
- `.plot(x=..., y=...)`: qué columna va en cada eje. Y ya.



La **curva de declive**: alta al inicio, cayendo con los años. Así viven y mueren los pozos.

Graficar un groupby: Barras

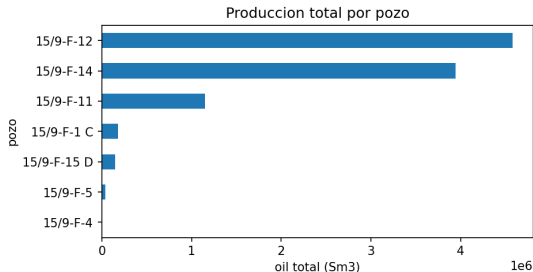
El resumen por pozo, ahora como gráfico de barras.

Python

```
totales = (prod.groupby("pozo")
           ["oil"].sum())

totales.plot(kind="barh")
```

- `kind="barh"`: barras horizontales. ("bar" = verticales, "line" = línea).
- ✓ Dos pozos (F-12 y F-14) produjeron **el 85 %** de todo el campo.



Su Turno: Operar y Graficar

🕒 10 min

Con la producción (prod) ya cargada:

1. ¿Cuál es la producción **promedio** de oil del campo? (`.mean()`)
2. ¿Y el **describe** completo de la columna gas?
3. Filtrar los días del pozo 15/9-F-14 (`==`).
4. Graficar la producción de F-14 contra la fecha (`.plot(x=..., y=...)`).
5. Con groupby, calcular el **agua** total por pozo y graficarla en barras.

Well Logs y el Formato LAS

El dato petrolero por excelencia

¿Qué es un Well Log (Registro de Pozo)?

Al perforar, se baja una herramienta que mide las propiedades de la roca a lo largo del pozo.

- ↓ El eje no es el tiempo: es la **profundidad** (metros).
- 📈 Cada propiedad medida es una **curva**: radioactividad, densidad, resistividad...
- 📊 La herramienta mide **cada pocos centímetros**: decenas de miles de registros por pozo.

Las curvas responden preguntas:

- ? ¿Qué **tipo de roca** hay a cada profundidad? ¿Arena o arcilla?
- ? ¿Tiene **poros** donde acumular petróleo?
- ? Esos poros, ¿tienen **petróleo o agua**?

Es el dato con el que se decide dónde completar y producir un pozo.

El Formato LAS: el Estándar de la Industria

LAS = Log ASCII Standard. Desde 1990, todo software petrolero lo lee y lo escribe.

- Es **texto**, como el CSV, pero con una estructura propia.
- ❗ Una **cabecera** describe el pozo: nombre, campo, operador, unidades...
- 📊 Después viene la **tabla de datos**: profundidad + una columna por curva.
- 🏠 Nuestro archivo: el pozo **15/9-19**, perforado por **Statoil** (hoy Equinor).

15-9-19.LAS (por dentro)

```
~Well Information
STRT.M    102.1568 : Top
STOP.M    4636.5140 : Bottom
STEP.M     .15240 : Increment
NULL.     -999.250 : Null Value
WELL.      15/9-19 : NAME
COMP.      STATOIL : OPERATOR
~Curve Information
DEPT.M      : DEPTH
GR  .GAPI : Gamma Ray
~ASCII
102.15  -999.25  -999.25
102.30  -999.25  -999.25
```

LAS vs CSV: ¿En Qué se Diferencian?

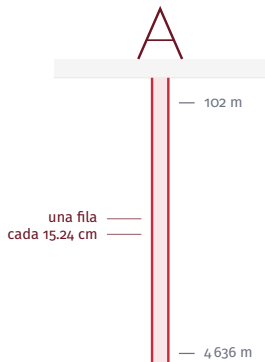
Los dos son texto y los dos son tablas. Pero cuentan cosas muy distintas.

	CSV (producción)	LAS (well log)
Cada fila es...	un día de un pozo	una profundidad del pozo
El “eje” es...	el tiempo (fechas)	la profundidad (metros)
Espaciado	un registro por día	un registro cada 15.24 cm
Metadatos	ninguno (solo encabezado)	cabecera completa: pozo, operador, unidades
Dato faltante	celda vacía	código -999.25
Quién lo usa	cualquier industria	estándar petrolero

💡 La buena noticia: una vez cargados en Python, **los dos se vuelven DataFrames** y se manejan igual.

Las Dimensiones del LAS: un Pozo en Números

Nuestro archivo mide el pozo 15/9-19 desde 102 m hasta 4 636 m de profundidad.



Dimensión

Valor real del archivo

Profundidad inicial	102.16 m
Profundidad final	4 636.51 m
Paso de medición	0.1524 m (= 6 pulgadas)
Filas (profundidades)	29 754
Columnas (curvas)	7

- En el CSV el índice era 0, 1, 2... Aquí el índice **es la profundidad**.
- 29 754 filas × 7 curvas ≈ **208 mil mediciones** en un solo pozo.

Las 7 Curvas: Qué nos Dice Cada Columna

Cada columna del LAS es un “sensor” distinto de la roca. Estas son las del pozo 15/9-19:

Curva	Unidad	Qué mide	Qué nos dice
GR	GAPI	radioactividad natural	arcilla (alta) vs arena (baja)
DEN	g/cm^3	densidad de la roca	porosidad
NEU	%	respuesta al neutrón	porosidad (hidrógeno)
AC	$\mu\text{s/ft}$	velocidad del sonido	porosidad, dureza
RDEP	$\text{ohm}\cdot\text{m}$	resistividad profunda	¿petróleo o agua?
RMED	$\text{ohm}\cdot\text{m}$	resistividad media	compara con RDEP
CALI	pulgadas	diámetro del hueco	calidad de la medición

💡 La clave de la resistividad: el agua salada **conduce** electricidad (resistividad baja); el petróleo **no conduce** (alta). Un RDEP alto en roca porosa = zona de interés.

lasio: la Librería para Leer LAS

Los tres pasos de toda librería: instalar → importar → usar.

Python

```
!pip install lasio

import lasio

las = lasio.read(url_las)
log = las.df()    # a DataFrame

print(log.shape)
print(list(log.columns))
```

>_ Output

```
(29754, 7)
['AC', 'CALI', 'DEN', 'GR',
 'NEU', 'RDEP', 'RMED']
```

- lasio **no viene** en Colab: por eso el pip install.
- .df() convierte el LAS en un **DataFrame**: cabecera afuera, curvas adentro.
- ✓ Los -999.25 se vuelven NaN automáticamente.

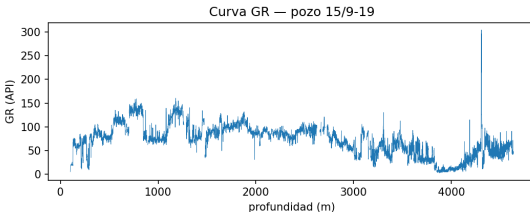
Graficar una Curva del Well Log

En el log, el índice es la profundidad: `.plot()` la pone automáticamente en el eje X.

Python

```
log["GR"].plot()
```

- Una sola línea: la Series GR contra su índice (la profundidad).
- Los picos de GR = zonas con más **arcilla**; los valles = **arenas** (posibles reservorios).
- i Los petrofísicos lo dibujan en **vertical**; lo aprenderemos más adelante en el diplomado.



Su Turno: Abrir el Well Log

🕒 7 min

Lean el LAS del pozo 15/9-19 y explórenlo como DataFrame.

En el notebook:

1. Instalar lasio con `!pip install lasio`.
2. Importarlo y leer el archivo desde la URL con `lasio.read(...)`.
3. Convertirlo a DataFrame con `.df()`.
4. ¿Cuántas filas tiene? ¿Qué curvas trae? (`.shape`, `.columns`)
5. Ver `.head()`: ¿por qué hay tantos NaN al inicio?
6. Sacar el `.describe()` de la curva GR y **graficarla** con `.plot()`.

💡 *Pista de la 5: las herramientas no miden desde la superficie; cada curva cubre solo un tramo del pozo.*

Limpieza de Datos

Breve pero esencial

Datos Faltantes: Contarlos Primero

Ya vimos los NaN. El primer paso siempre es saber cuántos hay y dónde.

Python

```
prod.isna().sum()
```

>_ Output

```
fecha      0
pozo       0
horas     285
oil       6473
gas       6473
agua      6473
```

- `.isna()` marca cada celda vacía; `.sum()` las cuenta por columna.
- **6473** días sin producción registrada (de 15 634): pozos aún no perforados o cerrados.
- ✓ Es **normal** en datos reales. Lo importante es decidir qué hacer.

Dos Remedios: Borrar o Rellenar

`dropna()` elimina las filas con huecos; `fillna()` los completa con un valor.

🗑 Borrar (con cuidado)

Python

```
limpio = prod.dropna()
print(prod.shape)
print(limpio.shape)
```

>_ Output

```
(15634, 7)
(9161, 7)
```

+ Rellenar (aquí, con 0)

Python

```
prod["oil"] =
    ↳ prod["oil"].fillna(0)
prod["agua"] =
    ↳ prod["agua"].fillna(0)
```

Pozo cerrado = 0 producción: aquí rellenar con 0 **tiene sentido físico**. En otros datos podría ser el promedio, o interpolar entre vecinos (`.interpolate()`), útil en logs).

💡 No hay receta única: borrar bota información; rellenar inventa valores. La decisión depende de **qué significa** el hueco.

Práctica Integradora: Reporte Volve

🕒 15 min

Todo lo de hoy, junto: el mismo reporte de la Clase 2, ahora con 15 634 filas reales.

1. **Cargar** el CSV de producción desde la URL.
2. **Contar** los faltantes por columna con `.isna().sum()`.
3. **Rellenar** con o los NaN de oil, gas y agua.
4. **Convertir** fecha con `pd.to_datetime`.
5. **Resumir**: producción total de oil por pozo (groupby). ¿Cuál produjo más?
6. **Graficar**: la producción del mejor pozo contra la fecha, y los totales en barras.

En la Clase 2, el reporte de 5 pozos nos tomó ~25 líneas. Hoy, con 15 634 filas, son ~8.

Cierre

Recapitulación

Lo Que Aprendimos Hoy

Conceptos

- ✓ Dato, dataset y datos tabulares
- ✓ Qué es una librería; `pip` e `import`
- ✓ Diccionarios: `{clave: valor}`
- ✓ `DataFrame` y `Series` (`pandas`)
- ✓ Qué es un well log y el formato **LAS**
- ✓ Qué mide cada curva (GR, DEN, RDEP...)
- ✓ Qué es un NaN y de dónde sale

Habilidades

- ✓ Cargar CSV por URL, subida o Drive
- ✓ Leer un LAS con `lasio`
- ✓ `head`, `shape`, `describe`, `unique`
- ✓ Filtrar filas y crear columnas
- ✓ Resumir con `groupby`
- ✓ Graficar con `.plot()` (línea y barras)
- ✓ Limpiar con `dropna` / `fillna`

Habilidad clave: tomar un archivo real, cargarlo, limpiarlo y sacar conclusiones.

Cierre del Bloque de Python

En 3 clases pasamos de cero a analizar datos petroleros reales.

- ➔ **Clase 1** — Fundamentos: variables, tipos, input, primeras decisiones.
- ➔ **Clase 2** — Control de flujo: if/elif/else, listas, ciclos for.
- ➔ **Clase 3** — pandas: librerías, carga de archivos reales (CSV y LAS), operaciones, gráficos y limpieza.
- ➔ **Lo que sigue** — profundizar el análisis y llegar a los **modelos predictivos** sobre estos mismos datos.

Ya pueden dirigir y auditar un análisis de datos: cargar, limpiar, resumir y graficar.

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Diplomado en Data Science Aplicada con Python

SLB Ecuador · UDLA · 2026